Desafio do PI — Container, TDD e Deploy automatizados

Unidade 2 · em grupos (os mesmos do Projeto Integrador) · entrega via Classroom, no prazo indicado na atividade

1. Contexto

Nas aulas de containers, CI/CD, TDD e BDD, o ola-java e o Rota Vital serviram de treino e de demonstração. Agora é a vez do projeto de vocês: o PI precisa subir do zero com um comando, ter as regras de negócio protegidas por testes escritos antes do código e chegar sozinho a um ambiente público a cada merge na main.

A pergunta central: **se um colega clonar o repositório amanhã, ele consegue rodar, testar e publicar o sistema sem pedir ajuda a ninguém?**

2. As três entregas

Uma entrega apoia a outra: o pipeline do E3 usa o Dockerfile do E1 e só publica se os testes do E2 passarem. Entreguem as três, nessa ordem de construção.

Entrega	O que é	Por quê
E1 · Container automatizado	O PI sobe do zero com um comando, igual em qualquer máquina.	É a base do deploy: o que roda no PC do grupo é o que vai para produção.
E2 · TDD	As regras do PI nascem dos testes, tarefa por tarefa.	Os testes são o gate da esteira: sem eles, o pipeline só empacota erro.
E3 · Deploy automatizado	Cada merge na main chega sozinho a um ambiente público.	Deploy manual é onde nascem os incidentes (Knight Capital, GitLab, AWS S3).

3. E1 · Container automatizado

O PI sobe do zero, com um comando, em qualquer máquina.

1. **Dockerfile multi-stage:** compila e roda os testes no 1º estágio; o 2º leva só o artefato.
2. **compose.yaml:** API + banco, com volume para os dados, healthcheck e depends_on; sobe com docker compose up -d --build.
3. **Configuração fora do código:** .env.example com as variáveis; nenhum segredo no repositório.
4. **README e evidências:** como subir do zero em poucos comandos; prints do docker compose ps e da API respondendo.

Justifiquem no README: a imagem base (tamanho, segurança), por que multi-stage, onde ficam os dados (volume) e como os serviços se encontram (rede do Compose).

4. E2 · TDD

Do story map às tarefas; de cada tarefa ao teste — escrito antes do código.

1. **Story map do PI:** backbone e Release 1; ao menos 3 histórias da Release 1 quebradas em tarefas (no README ou no board).
2. **Testes por nível:** unitário para cada tarefa de regra; ao menos 1 teste de integração (com o banco) e 1 de aceitação por história. O de aceitação pode ser um cenário BDD em Gherkin.

3. **Histórico red → green:** commits que mostram o teste antes do código (ex.: "test: …" seguido de "feat: …").
4. **Testes na esteira:** rodam a cada push e pull request; teste vermelho barra o merge.

Justifiquem no README: quais regras ganharam teste primeiro e por quê (risco), o que ficou sem teste e como a IA foi usada — declarada e revisada.

5. E3 · Deploy automatizado

Cada merge na main chega sozinho a um ambiente público — e só com a esteira verde.

1. **Pipeline completo:** build → testes → imagem → deploy, na ferramenta escolhida (GitHub Actions, GitLab CI, Jenkins, Azure Pipelines…).
2. **Fluxo de branch:** Git Flow, GitHub Flow ou GitLab Flow, aplicado de verdade: branches, pull requests e main protegida.
3. **Deploy automático:** para um serviço público (Render, Railway, Fly.io…), com os segredos no cofre da ferramenta.
4. **Evidências:** uma execução vermelha que barrou o deploy, uma verde que publicou, a URL no ar e como fazer rollback.

Justifiquem no README: a ferramenta de CI/CD, o fluxo de branch (por que esse e não outro), o provedor de deploy, onde ficam os segredos e como voltar à versão anterior.

6. Como justificar no README

Uma linha por decisão: o que escolheram, as alternativas e o porquê. Exemplos:

Decisão	Opções comuns	Exemplo de justificativa
Imagem base	node:22-alpine, python:3.12-slim, eclipse-temurin:21-jre…	"slim: menor e com menos pacotes vulneráveis; a compilação fica no 1º estágio."
Dados	PostgreSQL, MySQL, MongoDB; volume nomeado ou bind mount	"volume nomeado: os dados sobrevivem ao docker compose down."
Testes	qual regra testar primeiro	"compatibilidade ABO/Rh primeiro: um erro ali põe um paciente em risco."
Fluxo de branch	Git Flow, GitHub Flow, GitLab Flow	"GitHub Flow: grupo pequeno, uma versão no ar e deploy a cada merge."
CI/CD e deploy	GitHub Actions, GitLab CI; Render, Railway, Fly.io	"Actions: o código já está no GitHub; Render: plano gratuito com deploy por imagem."
Segredos	.env local; secrets da ferramenta	"senha do banco no GitHub Secrets; .env no .gitignore."

7. Entrega

Uma entrega por grupo, no Classroom, até o prazo da atividade:

1. **Link do repositório** (público, ou com acesso liberado para o professor).
2. **URL pública** do sistema no ar.
3. **README** com as seções: Integrantes · Como rodar · Testes · Pipeline e deploy · Decisões (a tabela da seção 6) · Uso de IA.
4. **Evidências** (prints ou links das execuções do pipeline) no README ou na pasta docs/evidencias.

Uso de IA: permitido e declarado — prompts principais no README. A IA rascunha; vocês validam e assinam.

8. Avaliação (10,0 pontos)

Critério	Pontos
E1 · Container automatizado: Dockerfile multi-stage, compose.yaml com volume e healthcheck, configuração fora do código, sobe do zero	3,0
E2 · TDD: story map e tarefas, testes por nível, histórico red → green, testes rodando na esteira	3,0
E3 · Deploy automatizado: pipeline com gates, fluxo de branch aplicado, deploy automático com segredos protegidos, evidências e rollback	3,0
Justificativas no README (tabela de decisões) e uso de IA declarado	1,0

Item sem evidência não pontua. Não entregou = 0.

9. Referências

- HUMBLE, J.; FARLEY, D. Continuous Delivery. Addison-Wesley, 2010 — cap. 5, o deployment pipeline.
- FOWLER, M. Continuous Integration. martinfowler.com, 2006.
- BECK, K. Test-Driven Development: By Example. Addison-Wesley, 2003.
- NORTH, D. Introducing BDD. Better Software, 2006.
- DOCKER DOCS. Compose file reference; Multi-stage builds. docs.docker.com.
- GITHUB DOCS. Understanding GitHub Actions; Building and testing Java with Maven. docs.github.com/actions.